VIDEO_CODE_EXAMPLES

HelixCode Phase 3 - Video Code Examples

Complete, runnable code examples for each video in the course.

Video 1-2: Setup and Getting Started

Basic Setup

```go
package main

import (
    "context"
    "fmt"
    "log"

    "dev.helix.code/internal/session"
    "dev.helix.code/internal/memory"
    "dev.helix.code/internal/template"
    "dev.helix.code/internal/persistence"
)

func main() {
    ctx := context.Background()

    // Initialize managers
    sessionMgr := session.NewManager()
    memoryMgr := memory.NewManager()
    templateMgr := template.NewManager()

    // Create persistence store
    store := persistence.NewStore("./helixcode_data")
    store.SetSessionManager(sessionMgr)
    store.SetMemoryManager(memoryMgr)
    store.SetTemplateManager(templateMgr)

    // Load existing state
```

```
    if err := store.Load(); err != nil {
        log.Printf("No existing state found: %v", err)
    }

    // Enable auto-save
    store.EnableAutoSave(300) // Every 5 minutes

    fmt.Println("HelixCode Phase 3 initialized successfully!")

    // Your code here...

    // Save on exit
    if err := store.Save(); err != nil {
        log.Fatal(err)
    }
}
```

Video 3-4: Session Management

Complete Session Example

```
package main

import (
    "context"
    "fmt"
    "time"

    "dev.helix.code/internal/session"
)

func main() {
    ctx := context.Background()
    mgr := session.NewManager()

    // Register callbacks
    mgr.OnCreate(func(s *session.Session) {
        fmt.Printf("Created: %s\n", s.Name)
    })

    mgr.OnStart(func(s *session.Session) {
        fmt.Printf("Started: %s\n", s.Name)
    })
```

```
mgr.OnComplete(func(s *session.Session) {
    duration := s.EndedAt.Sub(s.StartedAt)
    fmt.Printf("Completed: %s (Duration: %v)\n", s.Name,
    duration)
})

// Create and manage session
sess := mgr.Create("implement-api", session.ModeBuilding,
    "my-project")
sess.AddTag("api")
sess.AddTag("v2")
sess.SetMetadata("sprint", "24")
sess.SetContext("focus", "user-endpoints")

// Start work
mgr.Start(sess.ID)

// Simulate work
fmt.Println("Working on implementation...")
time.Sleep(2 * time.Second)

// Pause for break
mgr.Pause(sess.ID)
fmt.Println("Taking a break...")
time.Sleep(1 * time.Second)

// Resume
mgr.Resume(sess.ID)
fmt.Println("Back to work...")
time.Sleep(1 * time.Second)

// Complete
mgr.Complete(sess.ID)

// Query sessions
fmt.Println("\n--- Session Statistics ---")
stats := mgr.GetStatistics()
fmt.Printf("Total sessions: %d\n", stats.TotalSessions)
fmt.Printf("Completed: %d\n",
    stats.ByStatus[session.StatusCompleted])

// Export session
snapshot, _ := mgr.Export(sess.ID)
```

```
    fmt.Printf("\nExported session: %s\n", snapshot.Name)
}
```

Multi-Session Workflow

```
package main

import (
    "context"
    "fmt"

    "dev.helix.code/internal/session"
)

func main() {
    ctx := context.Background()
    mgr := session.NewManager()

    // Feature development with multiple sessions
    projects := []struct {
        name    string
        mode    session.Mode
        project string
    }{
        {"plan-auth", session.ModePlanning, "api"},
        {"implement-auth", session.ModeBuilding, "api"},
        {"test-auth", session.ModeTesting, "api"},
        {"plan-ui", session.ModePlanning, "frontend"},
        {"implement-ui", session.ModeBuilding, "frontend"},
    }

    // Create all sessions
    for _, p := range projects {
        sess := mgr.Create(p.name, p.mode, p.project)
        sess.AddTag("authentication")
        fmt.Printf("Created: %s (%s)\n", p.name, p.mode)
    }

    // Query examples
    fmt.Println("\n--- Queries ---")

    apiSessions := mgr.GetByProject("api")
    fmt.Printf("API sessions: %d\n", len(apiSessions))
```

```
    planningSessions := mgr.GetByMode(session.ModePlanning)
    fmt.Printf("Planning sessions: %d\n", len(planningSessions))

    authSessions := mgr.GetByTag("authentication")
    fmt.Printf("Auth-tagged sessions: %d\n", len(authSessions))
}
```

Video 5-6: Memory System

Conversation Management

```
package main

import (
    "context"
    "fmt"

    "dev.helix.code/internal/memory"
)

func main() {
    ctx := context.Background()
    mgr := memory.NewManager()

    // Create conversation
    conv := mgr.CreateConversation("JWT Implementation")
    conv.SessionID = "session-123"
    conv.SetMetadata("feature", "authentication")

    // Add system context
    mgr.AddMessage(conv.ID, memory.NewSystemMessage(
        "You are an expert Go developer specializing in
        authentication and security.",
    ))

    // Simulated conversation
    exchanges := []struct {
        role    string
        content string
    }{
        {"user", "How do I implement JWT authentication in Go?"},
```

```
        {"assistant", "Here's how to implement JWT
        authentication:\n\n1. Install the jwt-go package\n2.
        Create a signing key\n3. Generate tokens on login\n4.
        Validate tokens in middleware"},
        {"user", "Can you show me the code for generating a
        token?"},
        {"assistant", "Here's the token generation code:
        \n```go\nfunc GenerateToken(userID string) (string,
        error) {\n    claims := jwt.MapClaims{\n
        \"user_id\": userID,\n        \"exp\": time.Now().Add(24
        * time.Hour).Unix(),\n    }\n    token :=
        jwt.NewWithClaims(jwt.SigningMethodHS256, claims)\n
        return token.SignedString([]byte(secretKey))\n}\n```"},
        {"user", "How do I validate the token in middleware?"},
        {"assistant", "Here's the middleware:\n```go\nfunc
        AuthMiddleware(next http.Handler) http.Handler {\n
        return http.HandlerFunc(func(w http.ResponseWriter, r
        *http.Request) {\n        tokenString :=
        r.Header.Get(\"Authorization\")\n        token, err :=
        jwt.Parse(tokenString, func(t *jwt.Token) (interface{},
        error) {\n            return []byte(secretKey),
        nil\n        })\n        if err != nil || !token.Valid
        {\n            http.Error(w, \"Unauthorized\",
        http.StatusUnauthorized)\n            return\n        }
        \n        next.ServeHTTP(w, r)\n    })\n}\n```"},
}

for _, ex := range exchanges {
    var msg *memory.Message
    if ex.role == "user" {
        msg = memory.NewUserMessage(ex.content)
    } else {
        msg = memory.NewAssistantMessage(ex.content)
    }
    mgr.AddMessage(conv.ID, msg)
}

// Display statistics
stats := conv.GetStatistics()
fmt.Printf("Conversation: %s\n", conv.Title)
fmt.Printf("Messages: %d\n", stats.TotalMessages)
fmt.Printf("Tokens: %d\n", stats.TotalTokens)

// Search
results := conv.Search("middleware")
```

```
    fmt.Printf("\nSearch 'middleware': %d results\n",
        len(results))

    // Convert to text
    fmt.Println("\n--- Conversation Text ---")
    fmt.Println(conv.ToText())
}
```

Memory with Limits

```
package main

import (
    "context"
    "fmt"

    "dev.helix.code/internal/memory"
)

func main() {
    ctx := context.Background()
    mgr := memory.NewManager()

    conv := mgr.CreateConversation("Large Conversation Test")

    // Set limits
    conv.SetMaxMessages(50)
    conv.SetMaxTokens(10000)

    // Add many messages
    for i := 0; i < 100; i++ {
        msg := memory.NewUserMessage(fmt.Sprintf("Message number
        %d with some content", i))
        mgr.AddMessage(conv.ID, msg)
    }

    // Check result
    messages := conv.GetMessages()
    fmt.Printf("Messages after adding 100: %d (limit: 50)\n",
        len(messages))

    // Get recent
    recent := conv.GetRecent(10)
    fmt.Printf("Recent messages: %d\n", len(recent))
```

```
    // Manual truncate
    conv.Truncate(20)
    fmt.Printf("After truncate(20): %d messages\n",
        len(conv.GetMessages()))
}
```

Video 7-8: State Persistence

Complete Persistence Example

```
package main

import (
    "context"
    "fmt"
    "os"
    "time"

    "dev.helix.code/internal/session"
    "dev.helix.code/internal/memory"
    "dev.helix.code/internal/template"
    "dev.helix.code/internal/persistence"
)

func main() {
    ctx := context.Background()

    // Initialize managers
    sessionMgr := session.NewManager()
    memoryMgr := memory.NewManager()
    templateMgr := template.NewManager()

    // Create test data
    sess := sessionMgr.Create("test-session",
        session.ModeBuilding, "project")
    conv := memoryMgr.CreateConversation("Test Conversation")
    memoryMgr.AddMessage(conv.ID, memory.NewUserMessage("Test
        message"))

    tpl := template.NewTemplate("Test Template", "A test",
        template.TypeCode)
    tpl.SetContent("Hello {{name}}!")
    templateMgr.Register(tpl)
```

```
// Test all formats
formats := []struct {
    name   string
    format persistence.Format
}{
    {"JSON", persistence.FormatJSON},
    {"Compact JSON", persistence.FormatCompactJSON},
    {"GZIP", persistence.FormatJSONGZIP},
}

for _, f := range formats {
    dir := fmt.Sprintf("./test_persistence_%s", f.name)
    os.MkdirAll(dir, 0755)

    store := persistence.NewStore(dir)
    store.SetFormat(f.format)
    store.SetSessionManager(sessionMgr)
    store.SetMemoryManager(memoryMgr)
    store.SetTemplateManager(templateMgr)

    // Save
    start := time.Now()
    if err := store.Save(); err != nil {
        fmt.Printf("%s save error: %v\n", f.name, err)
        continue
    }
    saveTime := time.Since(start)

    // Check file size
    files, _ := os.ReadDir(dir)
    var totalSize int64
    for _, file := range files {
        info, _ := file.Info()
        totalSize += info.Size()
    }

    // Load
    start = time.Now()
    if err := store.Load(); err != nil {
        fmt.Printf("%s load error: %v\n", f.name, err)
        continue
    }
    loadTime := time.Since(start)
```

```
        fmt.Printf("%s: Size=%d bytes, Save=%v, Load=%v\n",
            f.name, totalSize, saveTime, loadTime)
    }
}
```

Auto-Save and Backup

```
package main

import (
    "context"
    "fmt"
    "time"

    "dev.helix.code/internal/session"
    "dev.helix.code/internal/persistence"
)

func main() {
    ctx := context.Background()

    sessionMgr := session.NewManager()
    store := persistence.NewStore("./data")
    store.SetSessionManager(sessionMgr)

    // Callbacks
    store.OnSave(func() {
        fmt.Printf("[%s] State saved\n",
        time.Now().Format("15:04:05"))
    })

    store.OnError(func(err error) {
        fmt.Printf("[ERROR] %v\n", err)
    })

    // Enable auto-save (every 10 seconds for demo)
    store.EnableAutoSave(10)
    fmt.Println("Auto-save enabled (10s interval)")

    // Create some data
    for i := 0; i < 5; i++ {
        sess := sessionMgr.Create(
            fmt.Sprintf("session-%d", i),
```

```
            session.ModeBuilding,
            "project",
        )
        fmt.Printf("Created session: %s\n", sess.Name)
        time.Sleep(3 * time.Second)
    }

    // Create backup
    backupPath := fmt.Sprintf("./backups/backup-%s",
        time.Now().Format("20060102-150405"))
    if err := store.Backup(backupPath); err != nil {
        fmt.Printf("Backup error: %v\n", err)
    } else {
        fmt.Printf("Backup created: %s\n", backupPath)
    }

    // Disable auto-save
    store.DisableAutoSave()
    fmt.Println("Auto-save disabled")
}
```

Video 9-10: Template System

Template Library

```
package main

import (
    "context"
    "fmt"

    "dev.helix.code/internal/template"
)

func main() {
    ctx := context.Background()
    mgr := template.NewManager()

    // HTTP Handler template
    httpHandler := template.NewTemplate(
        "HTTP Handler",
        "RESTful HTTP handler with common methods",
        template.TypeCode,
```

```
    )
    httpHandler.SetContent(`package handlers

import (
    "encoding/json"
    "net/http"
)

// {{HandlerName}}Handler handles {{Description}}
type {{HandlerName}}Handler struct {
    // Dependencies
}

func New{{HandlerName}}Handler() *{{HandlerName}}Handler {
    return &{{HandlerName}}Handler{}
}

func (h *{{HandlerName}}Handler) ServeHTTP(w http.ResponseWriter,
        r *http.Request) {
    switch r.Method {
    case http.MethodGet:
        h.handleGet(w, r)
    case http.MethodPost:
        h.handlePost(w, r)
    case http.MethodPut:
        h.handlePut(w, r)
    case http.MethodDelete:
        h.handleDelete(w, r)
    default:
        http.Error(w, "Method not allowed",
        http.StatusMethodNotAllowed)
    }
}

func (h *{{HandlerName}}Handler) handleGet(w http.ResponseWriter,
        r *http.Request) {
    // TODO: Implement GET
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(map[string]string{"status": "ok"})
}

func (h *{{HandlerName}}Handler) handlePost(w
        http.ResponseWriter, r *http.Request) {
    // TODO: Implement POST
}
```

```
func (h *{{HandlerName}}Handler) handlePut(w http.ResponseWriter,
        r *http.Request) {
    // TODO: Implement PUT
}

func (h *{{HandlerName}}Handler) handleDelete(w
        http.ResponseWriter, r *http.Request) {
    // TODO: Implement DELETE
}
`)
    httpHandler.AddVariable(template.Variable{
        Name:        "HandlerName",
        Description: "Name of the handler (e.g., User, Product)",
        Required:    true,
        Type:        "string",
    })
    httpHandler.AddVariable(template.Variable{
        Name:        "Description",
        Description: "What the handler does",
        Required:    true,
        Type:        "string",
    })
    httpHandler.AddTag("http")
    httpHandler.AddTag("handler")
    httpHandler.AddTag("rest")
    mgr.Register(httpHandler)

    // Test template
    testTemplate := template.NewTemplate(
        "Table Test",
        "Table-driven test file",
        template.TypeCode,
    )
    testTemplate.SetContent(`package {{Package}}_test

import (
    "testing"
)

func Test{{FunctionName}}(t *testing.T) {
    tests := []struct {
        name     string
        input    {{InputType}}
```

```
        expected {{OutputType}}
        wantErr  bool
    }{
        {
            name:     "valid input",
            input:    {{SampleInput}},
            expected: {{SampleOutput}},
            wantErr:  false,
        },
        // Add more test cases
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            got, err := {{FunctionName}}(tt.input)
            if (err != nil) != tt.wantErr {
                t.Errorf("{{FunctionName}}() error = %v, wantErr
        %v", err, tt.wantErr)
                return
            }
            if got != tt.expected {
                t.Errorf("{{FunctionName}}() = %v, want %v", got,
        tt.expected)
            }
        })
    }
}
`)
    testTemplate.AddTag("test")
    testTemplate.AddTag("table-driven")
    mgr.Register(testTemplate)

    // Register built-in templates
    mgr.RegisterBuiltinTemplates()

    // List all templates
    fmt.Println("--- Available Templates ---")
    for _, tpl := range mgr.GetAll() {
        fmt.Printf("- %s (%s): %s\n", tpl.Name, tpl.Type,
        tpl.Description)
    }

    // Render HTTP Handler
    fmt.Println("\n--- Generated HTTP Handler ---")
```

```
    result, err := httpHandler.Render(map[string]interface{}{
        "HandlerName": "User",
        "Description": "user CRUD operations",
    })
    if err != nil {
        fmt.Printf("Error: %v\n", err)
        return
    }
    fmt.Println(result)

    // Search templates
    fmt.Println("\n--- Search Results for 'http' ---")
    results := mgr.Search("http")
    for _, tpl := range results {
        fmt.Printf("- %s\n", tpl.Name)
    }
}
```

Video 11-12: Integration Patterns

Complete Development Workflow

```
package main

import (
    "context"
    "fmt"
    "time"

    "dev.helix.code/internal/session"
    "dev.helix.code/internal/memory"
    "dev.helix.code/internal/template"
    "dev.helix.code/internal/persistence"
)

func main() {
    ctx := context.Background()

    // Initialize all managers
    sessionMgr := session.NewManager()
    memoryMgr := memory.NewManager()
    templateMgr := template.NewManager()
```

```
// Setup persistence
store := persistence.NewStore("./project_data")
store.SetSessionManager(sessionMgr)
store.SetMemoryManager(memoryMgr)
store.SetTemplateManager(templateMgr)
store.EnableAutoSave(300)

// Register templates
templateMgr.RegisterBuiltinTemplates()

// Feature: Implement User API

// Phase 1: Planning
fmt.Println("=== Phase 1: Planning ===")
planSess := sessionMgr.Create("plan-user-api",
    session.ModePlanning, "api-project")
planSess.AddTag("user")
planSess.AddTag("api")
sessionMgr.Start(planSess.ID)

planConv := memoryMgr.CreateConversation("User API Planning")
planConv.SessionID = planSess.ID

memoryMgr.AddMessage(planConv.ID, memory.NewSystemMessage(
    "You are a software architect helping design a REST
    API.",
))
memoryMgr.AddMessage(planConv.ID, memory.NewUserMessage(
    "I need to design a User API with CRUD operations. What
    endpoints should I have?",
))
memoryMgr.AddMessage(planConv.ID, memory.NewAssistantMessage(
    "For a User API, I recommend these endpoints:\n"+
    "- GET /api/users - List all users\n"+
    "- GET /api/users/:id - Get specific user\n"+
    "- POST /api/users - Create user\n"+
    "- PUT /api/users/:id - Update user\n"+
    "- DELETE /api/users/:id - Delete user",
))

sessionMgr.Complete(planSess.ID)
fmt.Printf("Planning completed: %s\n", planSess.Name)

// Phase 2: Building
```

```
    fmt.Println("\n=== Phase 2: Building ===")
    buildSess := sessionMgr.Create("build-user-api",
        session.ModeBuilding, "api-project")
    buildSess.AddTag("user")
    buildSess.AddTag("api")
    sessionMgr.Start(buildSess.ID)

    buildConv := memoryMgr.CreateConversation("User API
        Implementation")
    buildConv.SessionID = buildSess.ID

    // Generate handler code using template
    handlerTemplate, _ := templateMgr.GetByName("Function")
    if handlerTemplate == nil {
        // Create simple function template if not found
        handlerTemplate = template.NewTemplate("Function", "Go
        function", template.TypeCode)
        handlerTemplate.SetContent(`func {{name}}({{params}})
        {{returns}} {
    {{body}}
}`)
        templateMgr.Register(handlerTemplate)
    }

    generatedCode, _ :=
        handlerTemplate.Render(map[string]interface{}{
        "name":    "GetUser",
        "params":  "w http.ResponseWriter, r *http.Request",
        "returns": "",
        "body":    "// Implementation here",
    })

    memoryMgr.AddMessage(buildConv.ID, memory.NewUserMessage(
        "Generate the GetUser handler",
    ))
    memoryMgr.AddMessage(buildConv.ID,
        memory.NewAssistantMessage(
        fmt.Sprintf("Here's the generated handler:
        \n```go\n%s\n```", generatedCode),
    ))

    sessionMgr.Complete(buildSess.ID)
    fmt.Printf("Building completed: %s\n", buildSess.Name)

    // Phase 3: Testing
```

```
fmt.Println("\n=== Phase 3: Testing ===")
testSess := sessionMgr.Create("test-user-api",
    session.ModeTesting, "api-project")
testSess.AddTag("user")
testSess.AddTag("api")
sessionMgr.Start(testSess.ID)

testConv := memoryMgr.CreateConversation("User API Testing")
testConv.SessionID = testSess.ID

memoryMgr.AddMessage(testConv.ID, memory.NewUserMessage(
    "What tests should I write for the User API?",
))
memoryMgr.AddMessage(testConv.ID, memory.NewAssistantMessage(
    "Key tests for User API:\n"+
    "1. TestGetUser_Success\n"+
    "2. TestGetUser_NotFound\n"+
    "3. TestCreateUser_Valid\n"+
    "4. TestCreateUser_InvalidInput\n"+
    "5. TestUpdateUser_Success\n"+
    "6. TestDeleteUser_Success",
))

sessionMgr.Complete(testSess.ID)
fmt.Printf("Testing completed: %s\n", testSess.Name)

// Summary
fmt.Println("\n=== Workflow Summary ===")

// Session statistics
stats := sessionMgr.GetStatistics()
fmt.Printf("Total sessions: %d\n", stats.TotalSessions)
fmt.Printf("Completed: %d\n",
    stats.ByStatus[session.StatusCompleted])

// Memory statistics
memStats := memoryMgr.GetStatistics()
fmt.Printf("Conversations: %d\n",
    memStats.TotalConversations)
fmt.Printf("Total messages: %d\n", memStats.TotalMessages)

// Save final state
if err := store.Save(); err != nil {
    fmt.Printf("Save error: %v\n", err)
```

```
    } else {
        fmt.Println("State saved successfully")
    }

    // Export sessions
    for _, sess := range sessionMgr.GetByTag("user") {
        snapshot, _ := sessionMgr.Export(sess.ID)
        fmt.Printf("Exported: %s\n", snapshot.Name)
    }
}
```

Usage Instructions

1. Copy the desired example to a Go file
2. Ensure you're in the HelixCode project directory
3. Run with `go run <filename>.go`
4. Modify and experiment with the code

For questions, refer to the video course or documentation.